

ЛАБОРАТОРНАЯ РАБОТА № 4

«Наследование. Обработка исключительных ситуаций»

Студент(ка): Зырянова Татьяна Евгеньевна

Группа: РИМ – 150950

Предмет: Программирование на Java

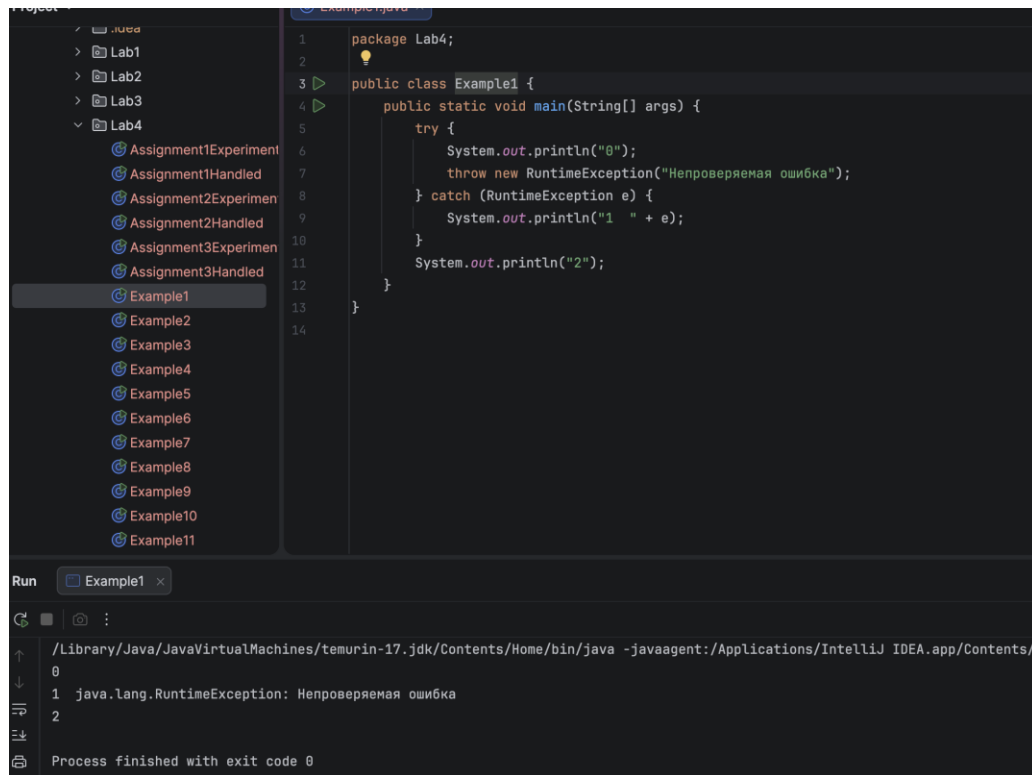
Репозиторий ДЗ: https://github.com/ZyrT12/java-core_2026-2027

Цель: знакомство с иерархией классов исключений и получение навыков обработки ошибок.

Описание задачи: в данной лабораторной работе необходимо изучить наследование, иерархию классов исключений и способы обработки исключительных ситуаций в языке Java. В ходе выполнения работы требуется воспроизвести примеры 1–14 лабораторной работы, исправить ошибки в примерах, выполнить задания из таблицы 2, определить классы возникающих исключений, создать обработчики ошибок и решить две задачи с сайта Timus.

Ход выполнения работы

Пример 1. Сгенерировано и перехвачено RuntimeException.



```
1 package Lab4;
2
3 public class Example1 {
4     public static void main(String[] args) {
5         try {
6             System.out.println("8");
7             throw new RuntimeException("Непроверяемая ошибка");
8         } catch (RuntimeException e) {
9             System.out.println("1 " + e);
10        }
11        System.out.println("2");
12    }
13 }
14
```

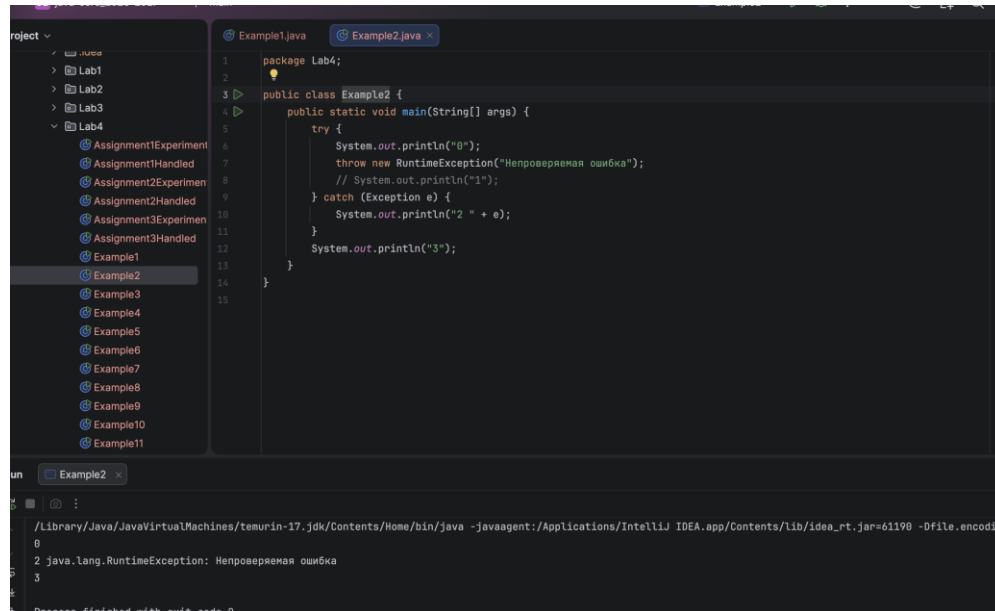
Run Example1 x

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/
0
1 java.lang.RuntimeException: Непроверяемая ошибка
2
Process finished with exit code 0
```

Рисунок 1. Результат работы программы Example1

Пример 2. Исключение перехвачено перехватчиком предка.

В исходном примере после оператора throw присутствовала строка, которая не могла быть выполнена. Для корректной компиляции она закомментирована.

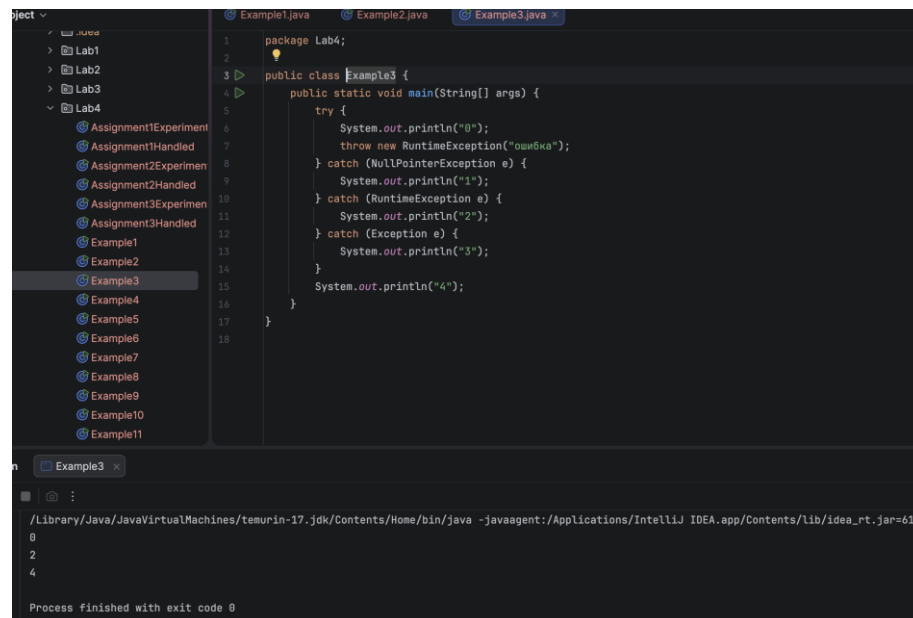


```
1 package Lab4;
2
3 public class Example2 {
4     public static void main(String[] args) {
5         try {
6             System.out.println("0");
7             throw new RuntimeException("Непроверяемая ошибка");
8             // System.out.println("1");
9         } catch (Exception e) {
10            System.out.println("2 " + e);
11        }
12        System.out.println("3");
13    }
14 }
15
```

```
/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=61190 -Dfile.encoding=UTF-8
0
2 java.lang.RuntimeException: Непроверяемая ошибка
3
Process finished with exit code 0
```

Рисунок 2. Результат работы программы Example2

Пример 3. Перехват исключения подходящим классом.



```
1 package Lab4;
2
3 public class Example3 {
4     public static void main(String[] args) {
5         try {
6             System.out.println("0");
7             throw new RuntimeException("ошибка");
8         } catch (NullPointerException e) {
9             System.out.println("1");
10        } catch (RuntimeException e) {
11            System.out.println("2");
12        } catch (Exception e) {
13            System.out.println("3");
14        }
15        System.out.println("4");
16    }
17 }
18
```

```
/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=6122 -Dfile.encoding=UTF-8
0
2
4
Process finished with exit code 0
```

Рисунок 3. Результат работы программы Example3

Пример 4. Перехват исключения подходящим классом.

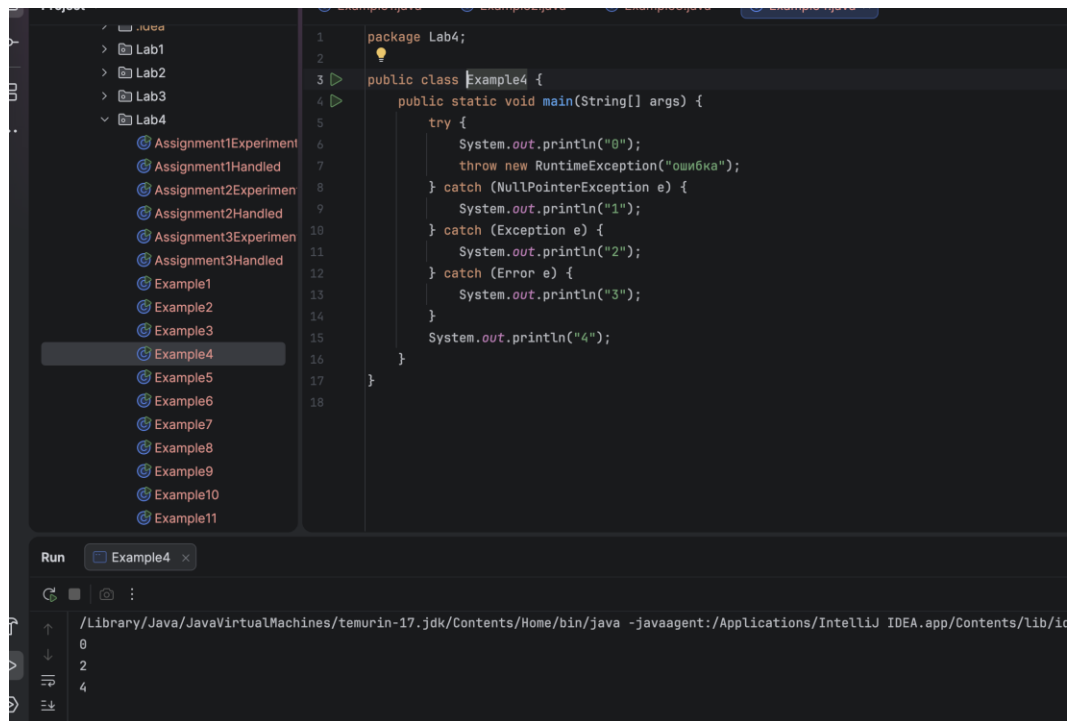


Рисунок 4. Результат работы программы Example4

Пример 5. Исключение не перехвачено.

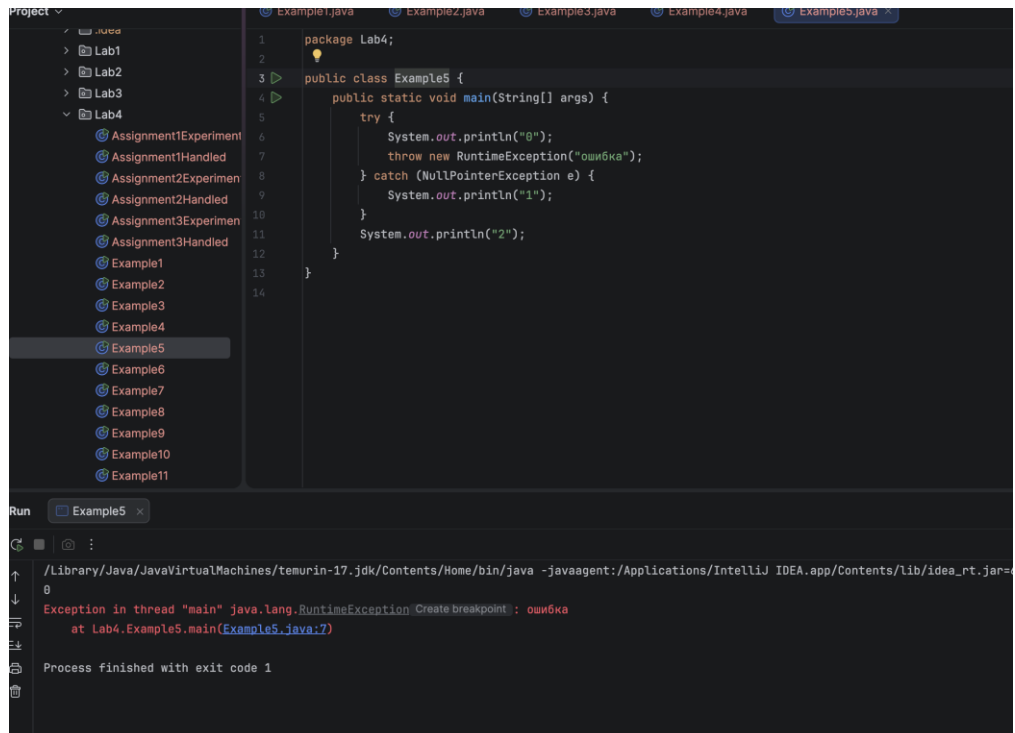
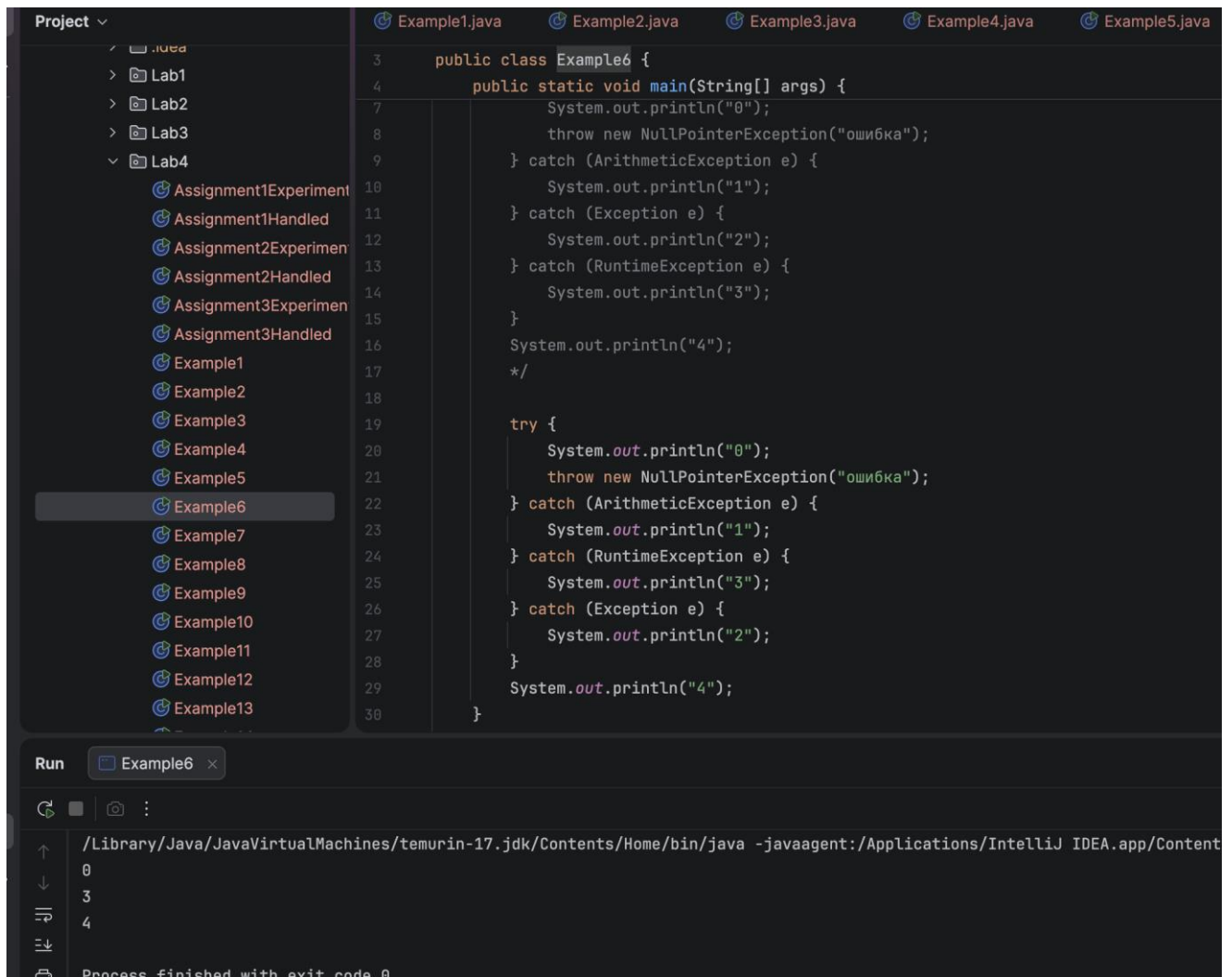


Рисунок 5. Результат работы программы Example5

Пример 6. Последовательность перехвата должна соответствовать иерархии классов исключений. Предок не должен перехватывать исключения раньше потомков.

В исходном примере порядок catch-блоков нарушал иерархию исключений: предок Exception был расположен раньше потомка RuntimeException. В исправленной версии сначала перехватываются более конкретные исключения, затем более общие.



The screenshot displays the IntelliJ IDEA IDE. On the left, the 'Project' view shows a directory structure with 'Lab4' containing several files, including 'Example6' which is selected. The main editor window shows the source code of 'Example6.java'. The code defines a public class 'Example6' with a 'main' method. It prints '0' and throws a 'NullPointerException' with the message 'ошибка'. It then enters a 'try' block where it prints '0', throws another 'NullPointerException' with the same message, and is followed by three 'catch' blocks: 'ArithmeticException' (prints '1'), 'RuntimeException' (prints '3'), and 'Exception' (prints '2'). After the try-catch block, it prints '4'. The 'Run' toolbar at the bottom shows 'Example6' as the active configuration. The output console at the bottom shows the execution path and the sequence of printed numbers: 0, 3, 4, followed by the message 'Process finished with exit code 0'.

```
3 public class Example6 {
4     public static void main(String[] args) {
5         System.out.println("0");
6         throw new NullPointerException("ошибка");
7     } catch (ArithmeticException e) {
8         System.out.println("1");
9     } catch (Exception e) {
10        System.out.println("2");
11    } catch (RuntimeException e) {
12        System.out.println("3");
13    }
14    System.out.println("4");
15    */
16
17    try {
18        System.out.println("0");
19        throw new NullPointerException("ошибка");
20    } catch (ArithmeticException e) {
21        System.out.println("1");
22    } catch (RuntimeException e) {
23        System.out.println("3");
24    } catch (Exception e) {
25        System.out.println("2");
26    }
27    System.out.println("4");
28 }
29
30 }
```

Run Example6 x

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Content

0
3
4
Process finished with exit code 0

Рисунок 6. Результат работы программы Example6

Пример 7. Нельзя перехватить брошенное исключение с помощью чужого catch, даже если перехватчик подходит.

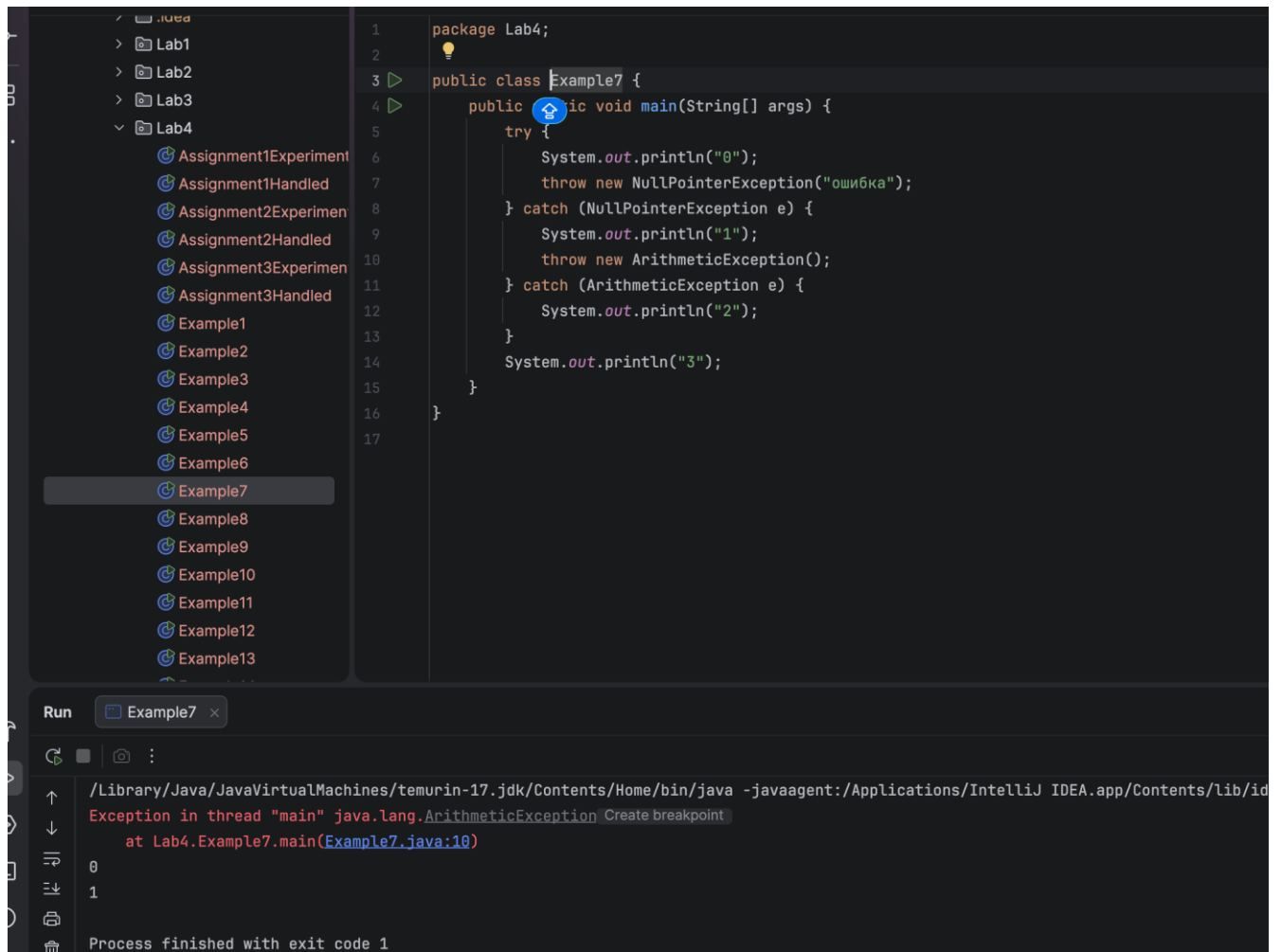


Рисунок 7. Результат работы программы Example7

Пример 8. Генерация исключения в методе.

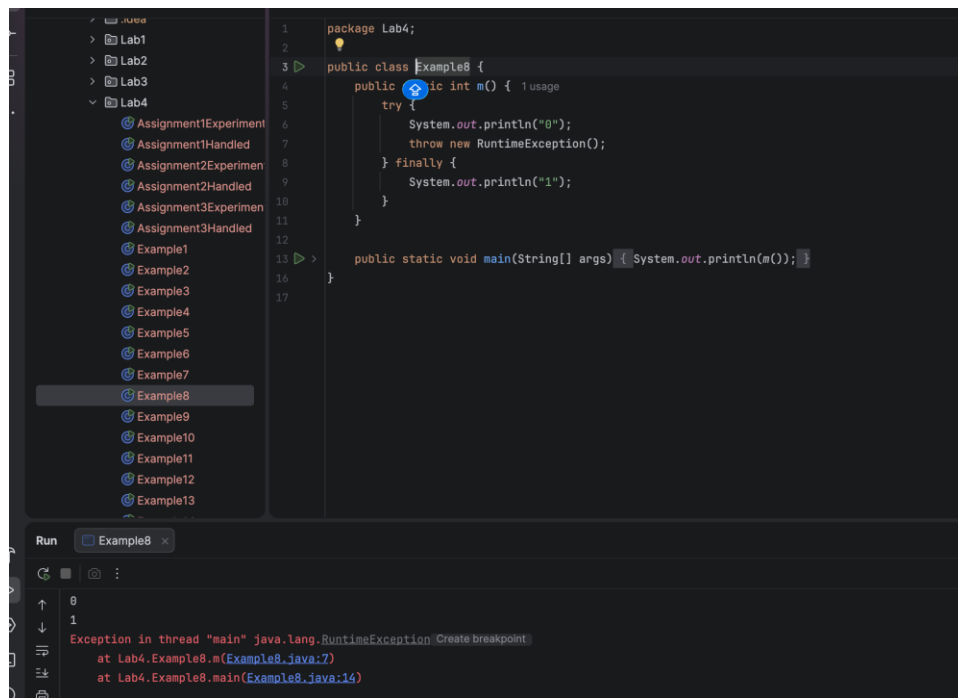


Рисунок 8. Результат работы программы Example8

Пример 9. Генерация исключительной ситуации в методе и дополнительное использование оператора return.

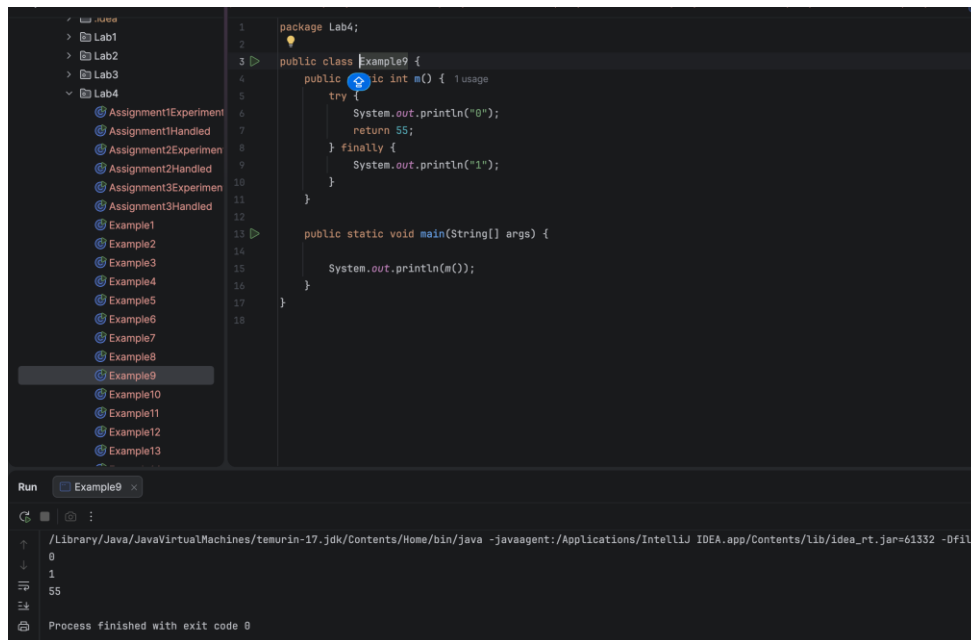
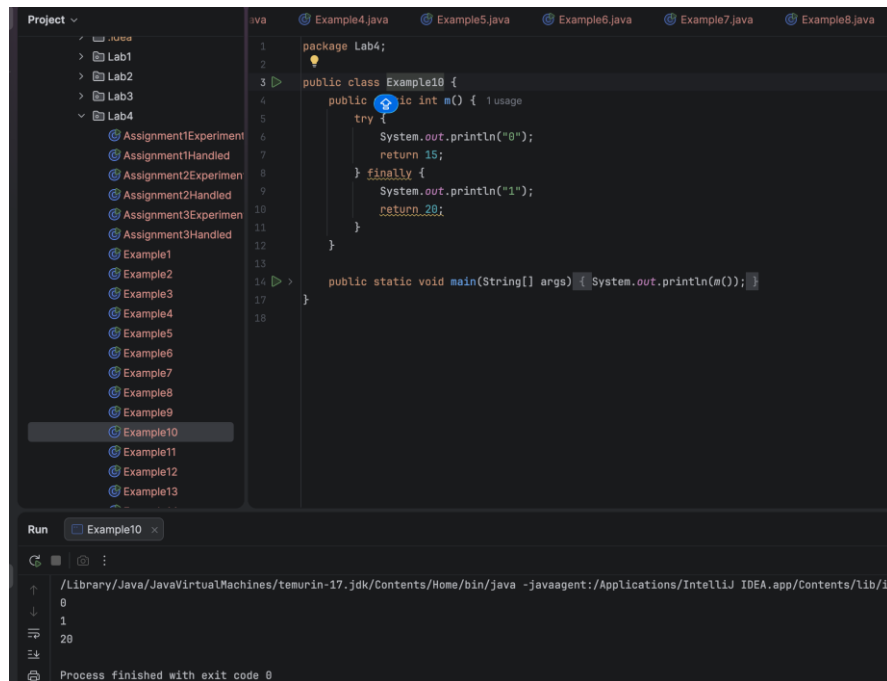


Рисунок 9. Результат работы программы Example9

Пример 10. Генерация исключительной ситуации в методе. Использование оператора return в секциях try и finally.



The screenshot shows the IntelliJ IDEA interface. On the left, the 'Project' view displays a directory structure with 'Lab4' containing various files, including 'Example10'. The main editor window shows the code for 'Example10.java' in the 'package Lab4;' namespace. The code defines a public class 'Example10' with a public method 'm()' that returns an 'int'. The method body contains a 'try' block with 'System.out.println("0");' and 'return 15;', followed by a 'finally' block with 'System.out.println("1");' and 'return 20;'. A 'main' method is also present, which calls 'm()' and prints the result. The 'Run' window at the bottom shows the execution output: '0' followed by '20', indicating that the 'finally' block's return statement overrode the 'try' block's return value.

```
1 package Lab4;
2
3 public class Example10 {
4     public int m() { 1 usage
5         try {
6             System.out.println("0");
7             return 15;
8         } finally {
9             System.out.println("1");
10            return 20;
11        }
12    }
13
14    public static void main(String[] args) { System.out.println(m()); }
15
16 }
```

Run Example10 x

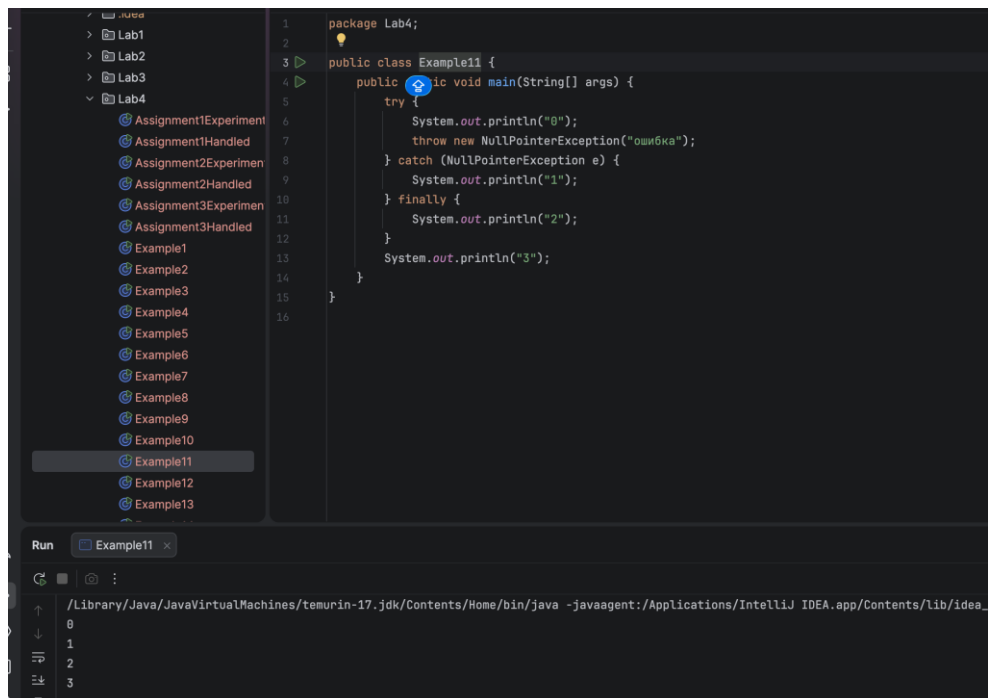
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_

0
1
20

Process finished with exit code 0

Рисунок 10. Результат работы программы Example10

Пример 11. Использование конструкции try-catch-finally.



The screenshot shows the IntelliJ IDEA interface. On the left, the 'Project' view displays a directory structure with 'Lab4' containing various files, including 'Example11'. The main editor window shows the code for 'Example11.java' in the 'package Lab4;' namespace. The code defines a public class 'Example11' with a public method 'main(String[] args)'. The method body contains a 'try' block with 'System.out.println("0");' and 'throw new NullPointerException("ошибка");', followed by a 'catch' block for 'NullPointerException' with 'System.out.println("1");', and a 'finally' block with 'System.out.println("2");'. After the 'finally' block, there is a 'System.out.println("3");' statement. The 'Run' window at the bottom shows the execution output: '0' followed by '2' followed by '3', indicating that the exception was caught and the 'finally' block executed successfully.

```
1 package Lab4;
2
3 public class Example11 {
4     public void main(String[] args) {
5         try {
6             System.out.println("0");
7             throw new NullPointerException("ошибка");
8         } catch (NullPointerException e) {
9             System.out.println("1");
10        } finally {
11            System.out.println("2");
12        }
13        System.out.println("3");
14    }
15
16 }
```

Run Example11 x

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_

0
1
2
3

Рисунок 11. Результат работы программы Example11

Пример 12. Исключение `IllegalArgumentException` – неверные аргументы.

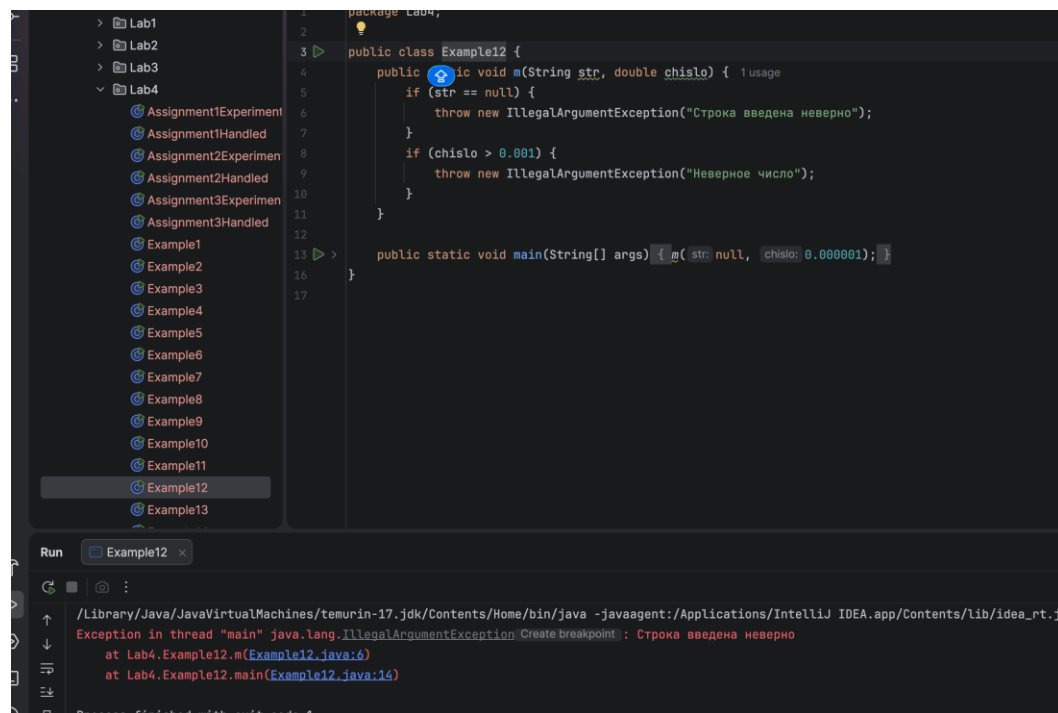


Рисунок 12. Результат работы программы `Example12`

Пример 13. Пример работы с аргументами метода `main`.

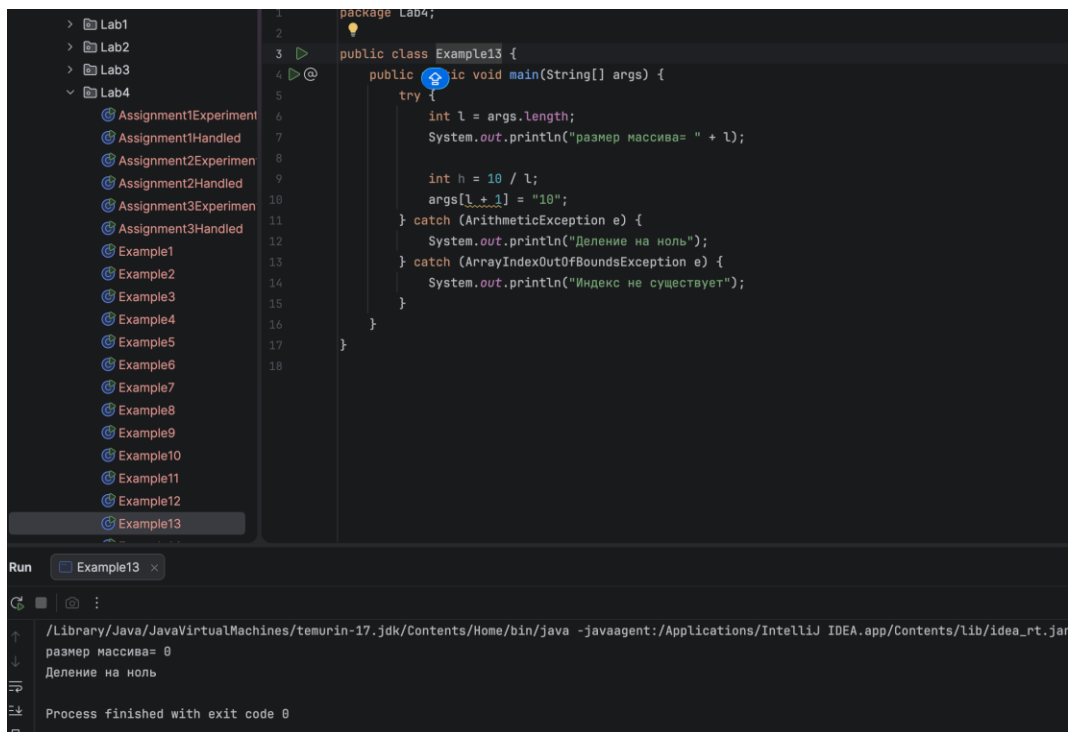
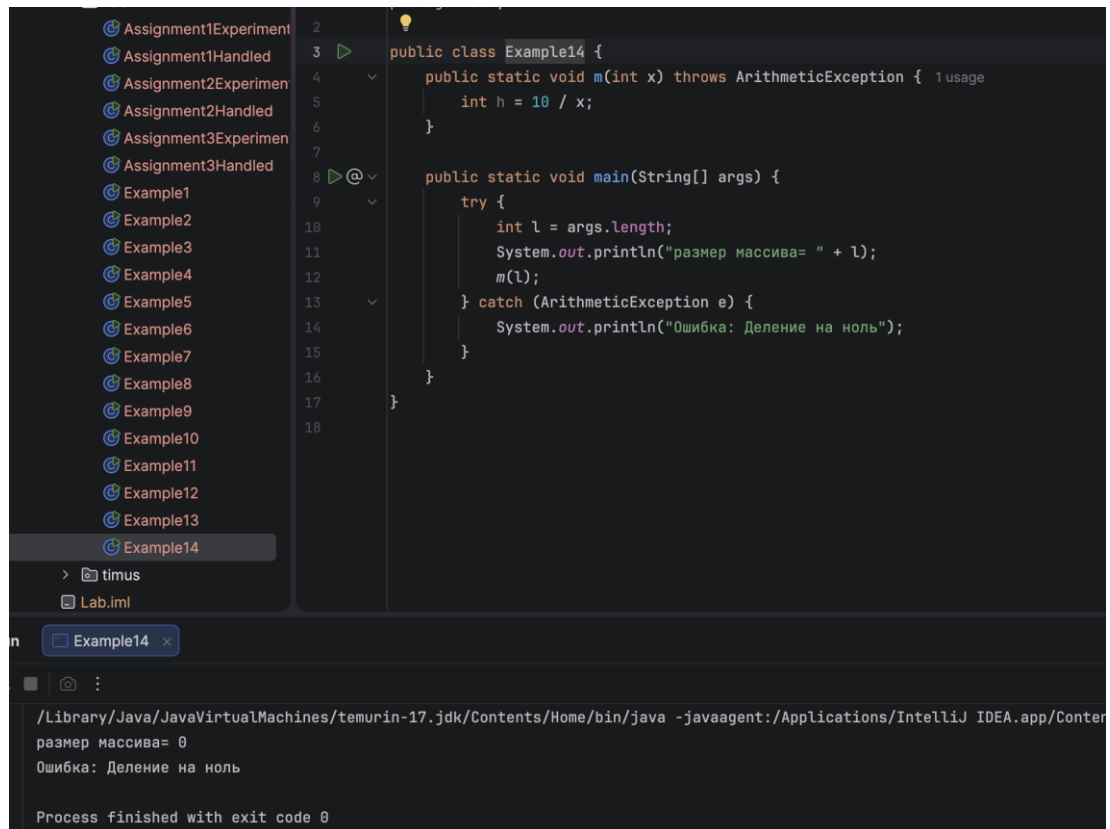


Рисунок 13. Результат работы программы `Example13`

Пример 14. Обработка исключения, порожденного одним методом `m()` в другом, в методе `main`.



The screenshot displays an IDE with a project explorer on the left containing various files like `Assignment1Experiment`, `Assignment1Handled`, and several `Example` files. The main editor shows the source code for `Example14`:

```
2 public class Example14 {
3     public static void m(int x) throws ArithmeticException { 1 usage
4         int h = 10 / x;
5     }
6
7     public static void main(String[] args) {
8         try {
9             int l = args.length;
10            System.out.println("размер массива= " + l);
11            m(l);
12        } catch (ArithmeticException e) {
13            System.out.println("Ошибка: Деление на ноль");
14        }
15    }
16 }
17
18
```

Below the code editor, the output console shows the results of running the program:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Conte
размер массива= 0
Ошибка: Деление на ноль
Process finished with exit code 0
```

Рисунок 14. Результат работы программы `Example14`

Задание 1. Воспроизвести примеры 1-14 лабораторной работы и при отправке в удаленный репозиторий исправить ошибки

Задание 2. Выполнить все задания из таблицы2:

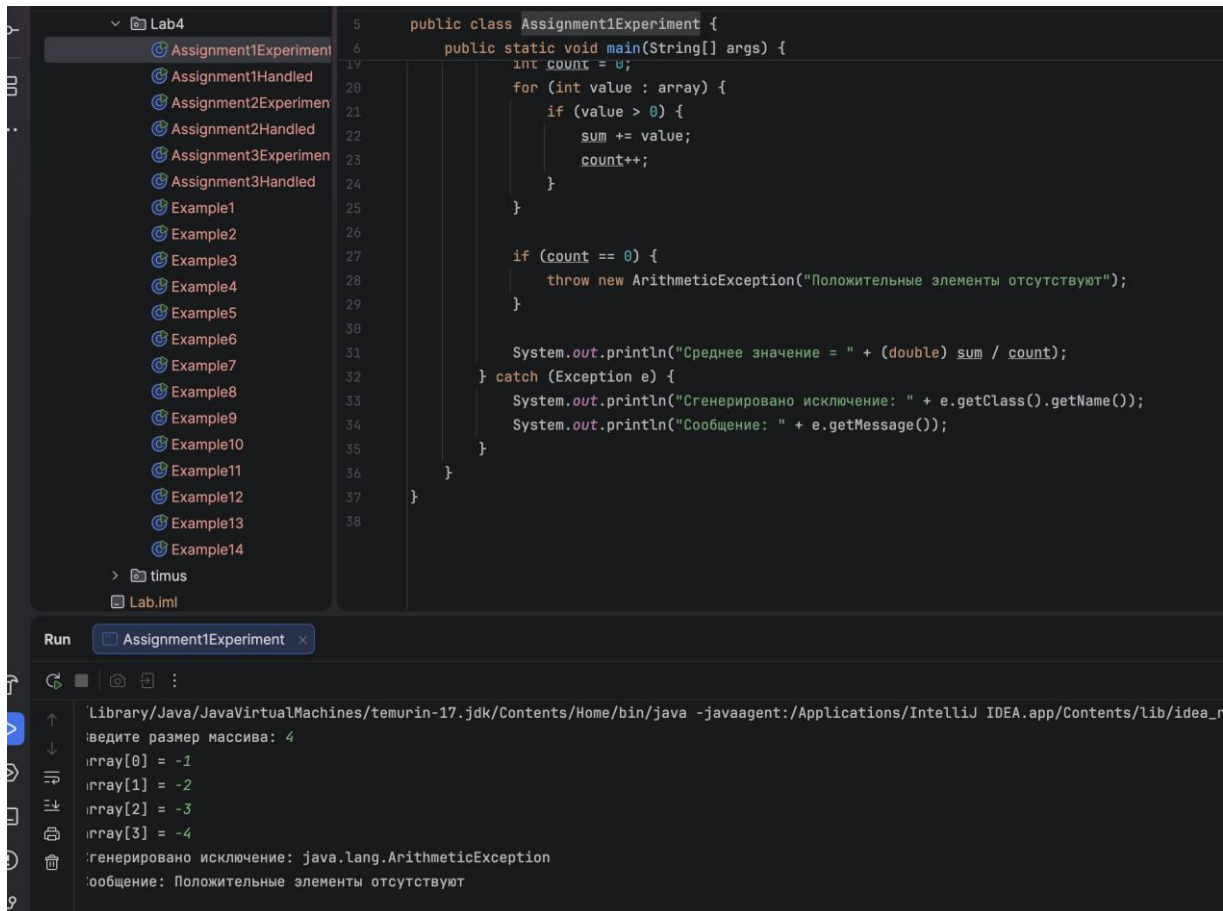
- определить экспериментально, ошибки каких классов будут сгенерированы;
- создать обработчики исключительных ситуаций с использованием выявленных классов и всех секций конструкции обработчика с соответствующими сообщениями, позволяющими корректно выполнить программу.

Таблица 1

Задание 1 В программе, вычисляющей среднее значение среди положительных элементов одномерного массива (тип элементов <code>int</code>), вводимого с клавиатуры, могут возникать ошибки в следующих случаях: – ввод строки вместо числа; – несоответствие числового типа данных; – положительные элементы отсутствуют.
Задание 2 В программе, где требуется из матрицы вывести столбец с номером, заданным с клавиатуры, могут возникать ошибки в следующих случаях: – ввод строки вместо числа; – нет столбца с таким номером.
Задание 3 В программе, вычисляющей сумму элементов типа <code>byte</code> одномерного массива, вводимого с клавиатуры, могут возникать ошибки в следующих случаях: – ввод строки вместо числа; – ввод или вычисление значения за границами диапазона типа.

Задание 1. Вычисление среднего значения среди положительных элементов одномерного массива.

На рисунке 15 показан результат экспериментального определения ошибок в задании 1.



The screenshot displays the IntelliJ IDEA IDE. The left sidebar shows a project named 'Lab4' with a list of files including 'Assignment1Experiment', 'Assignment1Handled', 'Assignment2Experiment', 'Assignment2Handled', 'Assignment3Experiment', 'Assignment3Handled', and several 'Example' files. The main editor window shows the source code of 'Assignment1Experiment.java'. The code defines a public class with a static void main method that takes a String array as an argument. It initializes a count variable to 0 and iterates through the array, summing positive values and incrementing the count. If the count remains 0, it throws an ArithmeticException with the message 'Положительные элементы отсутствуют'. Otherwise, it prints the average value. A try-catch block handles the exception, printing the class name and message. The bottom panel shows the 'Run' output for 'Assignment1Experiment', displaying the command used to run the program, the input '4', the array values [-1, -2, -3, -4], the generated exception 'java.lang.ArithmeticException', and the message 'Положительные элементы отсутствуют'.

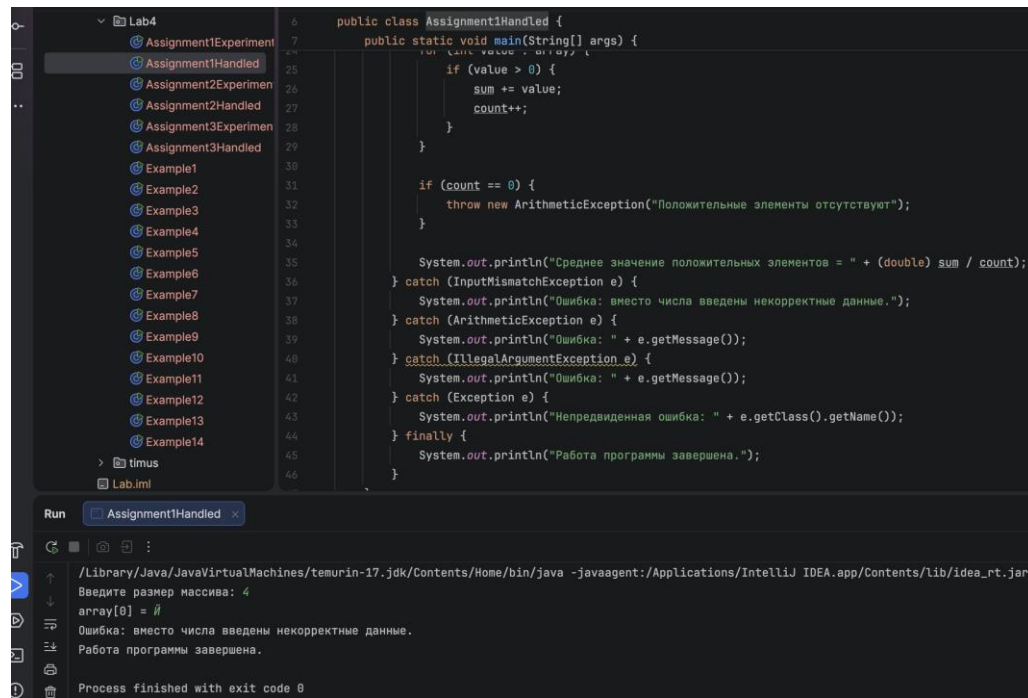
```
5 public class Assignment1Experiment {
6     public static void main(String[] args) {
7         int count = 0;
8         for (int value : array) {
9             if (value > 0) {
10                 sum += value;
11                 count++;
12             }
13         }
14
15         if (count == 0) {
16             throw new ArithmeticException("Положительные элементы отсутствуют");
17         }
18
19         System.out.println("Среднее значение = " + (double) sum / count);
20     } catch (Exception e) {
21         System.out.println("Сгенерировано исключение: " + e.getClass().getName());
22         System.out.println("Сообщение: " + e.getMessage());
23     }
24 }
25
26
27
28
29
30
31
32
33
34
35
36
37
38
```

Run Assignment1Experiment x

Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_r...
Введите размер массива: 4
array[0] = -1
array[1] = -2
array[2] = -3
array[3] = -4
Сгенерировано исключение: java.lang.ArithmeticException
Сообщение: Положительные элементы отсутствуют

Рисунок 15. Экспериментальное определение ошибок задания 1

На рисунке 16 показан результат выполнения программы с обработчиками исключений для задания 1.



The screenshot displays an IDE with a project named 'Lab4'. The left sidebar shows a list of files, including 'Assignment1Handled'. The main editor window shows the source code for 'Assignment1Handled.java'. The code defines a public class with a main method that processes an array of integers. It includes exception handling for 'InputMismatchException', 'ArithmeticException', and 'IllegalArgumentException'. The output window at the bottom shows the execution results, including the prompt 'Введите размер массива: 4', the input 'array[0] = 0', the error message 'Ошибка: вместо числа введены некорректные данные.', and the final message 'Работа программы завершена.'.

```
6 public class Assignment1Handled {
7     public static void main(String[] args) {
25         for (int value : args) {
26             if (value > 0) {
27                 sum += value;
28                 count++;
29             }
30
31             if (count == 0) {
32                 throw new ArithmeticException("Положительные элементы отсутствуют");
33             }
34
35             System.out.println("Среднее значение положительных элементов = " + (double) sum / count);
36         } catch (InputMismatchException e) {
37             System.out.println("Ошибка: вместо числа введены некорректные данные.");
38         } catch (ArithmeticException e) {
39             System.out.println("Ошибка: " + e.getMessage());
40         } catch (IllegalArgumentException e) {
41             System.out.println("Ошибка: " + e.getMessage());
42         } catch (Exception e) {
43             System.out.println("Непредвиденная ошибка: " + e.getClass().getName());
44         } finally {
45             System.out.println("Работа программы завершена.");
46         }
47     }
48 }
```

Run Assignment1Handled

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=

Введите размер массива: 4

array[0] = 0

Ошибка: вместо числа введены некорректные данные.

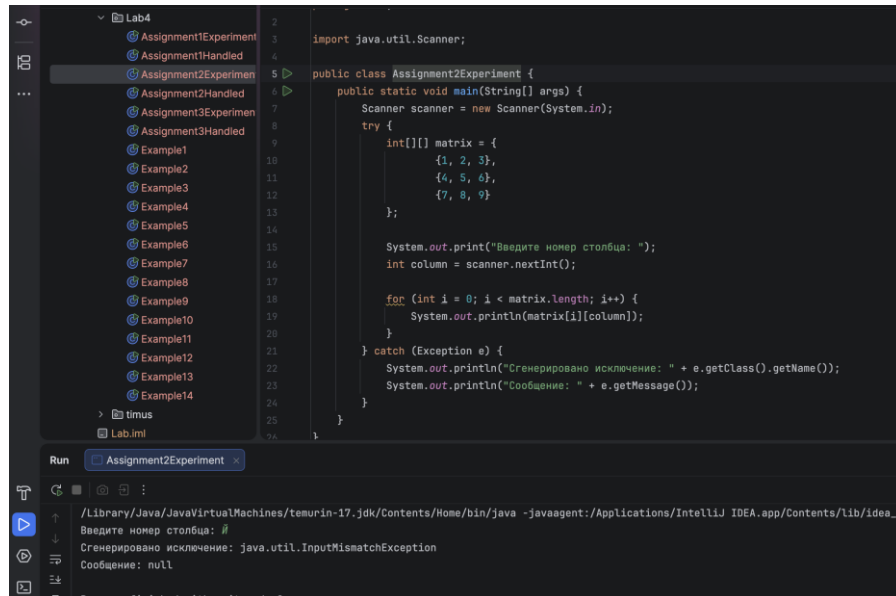
Работа программы завершена.

Process finished with exit code 0

Рисунок 16. Результат выполнения программы с обработчиками задания 1

Задание 2. Вывод столбца матрицы с номером, заданным с клавиатуры.

На рисунке 17 показан результат экспериментального определения ошибок в задании 2.



```
import java.util.Scanner;

public class Assignment2Experiment {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            int[][] matrix = {
                {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9}
            };

            System.out.print("Введите номер столбца: ");
            int column = scanner.nextInt();

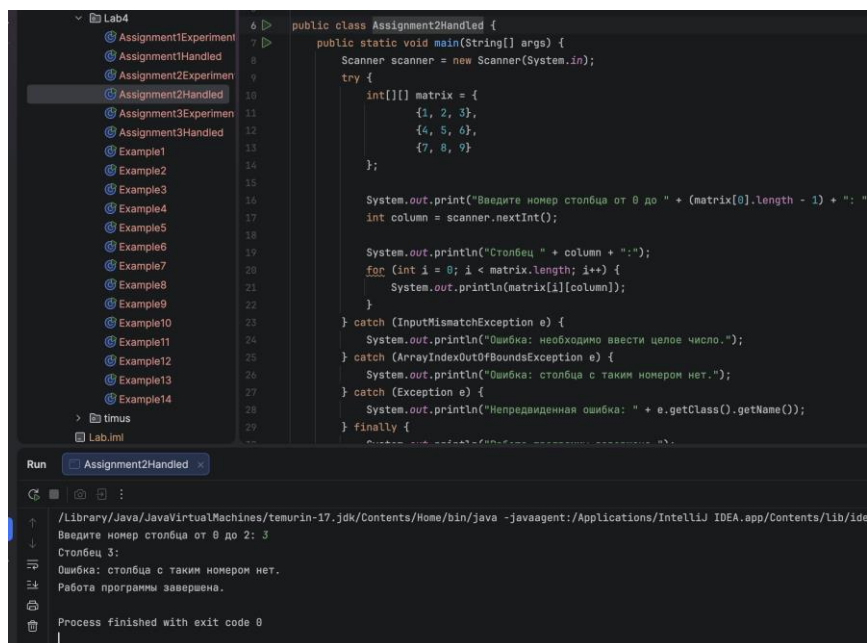
            for (int i = 0; i < matrix.length; i++) {
                System.out.println(matrix[i][column]);
            }
        } catch (Exception e) {
            System.out.println("Сгенерировано исключение: " + e.getClass().getName());
            System.out.println("Сообщение: " + e.getMessage());
        }
    }
}
```

Run Assignment2Experiment

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_r...
Введите номер столбца: #
Сгенерировано исключение: java.util.InputMismatchException
Сообщение: null
Process finished with exit code 0

Рисунок 17. Экспериментальное определение ошибок задания 2

На рисунке 18 показан результат выполнения программы с обработчиками исключений для задания 2.



```
public class Assignment2Handled {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            int[][] matrix = {
                {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9}
            };

            System.out.print("Введите номер столбца от 0 до " + (matrix[0].length - 1) + ": ");
            int column = scanner.nextInt();

            System.out.println("\nСтолбец " + column + ":");
            for (int i = 0; i < matrix.length; i++) {
                System.out.println(matrix[i][column]);
            }
        } catch (InputMismatchException e) {
            System.out.println("Ошибка: необходимо ввести целое число.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Ошибка: столбец с таким номером нет.");
        } catch (Exception e) {
            System.out.println("Непредвиденная ошибка: " + e.getClass().getName());
        } finally {
            System.out.println("Работа программы завершена.");
        }
    }
}
```

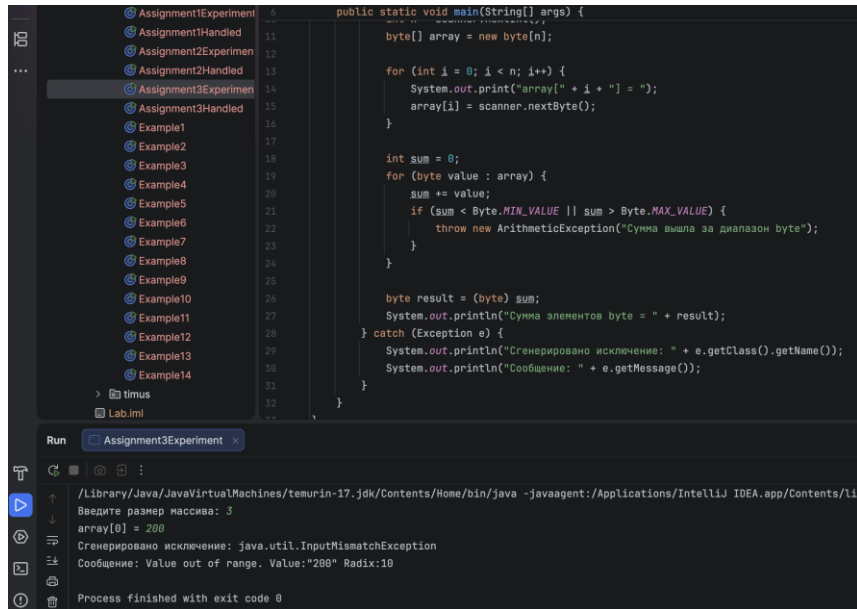
Run Assignment2Handled

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/ide...
Введите номер столбца от 0 до 2: 3
Столбец 3:
Ошибка: столбец с таким номером нет.
Работа программы завершена.
Process finished with exit code 0

Рисунок 18. Результат выполнения программы с обработчиками задания 2

Задание 3. Вычисление суммы элементов типа byte одномерного массива.

На рисунке 19 показан результат экспериментального определения ошибок в задании 3.



```
public static void main(String[] args) {  
    byte[] array = new byte[n];  
  
    for (int i = 0; i < n; i++) {  
        System.out.print("array[" + i + "] = ");  
        array[i] = scanner.nextByte();  
    }  
  
    int sum = 0;  
    for (byte value : array) {  
        sum += value;  
        if (sum < Byte.MIN_VALUE || sum > Byte.MAX_VALUE) {  
            throw new ArithmeticException("Сумма вышла за диапазон byte");  
        }  
    }  
  
    byte result = (byte) sum;  
    System.out.println("Сумма элементов byte = " + result);  
} catch (Exception e) {  
    System.out.println("Сгенерировано исключение: " + e.getClass().getName());  
    System.out.println("Сообщение: " + e.getMessage());  
}
```

Run Assignment3Experiment

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib

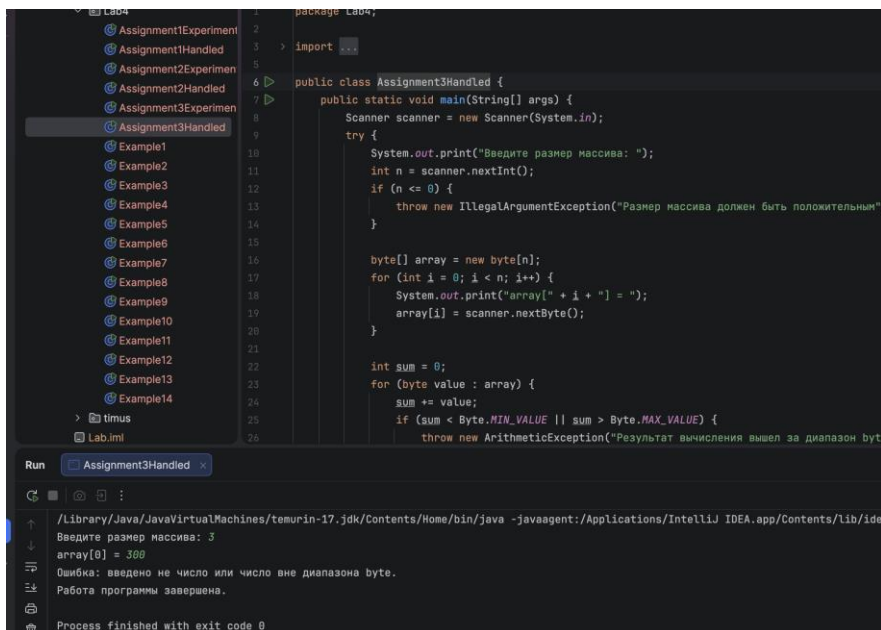
Введите размер массива: 3
array[0] = 200

Сгенерировано исключение: java.util.InputMismatchException
Сообщение: Value out of range. Value:"200" Radix:10

Process finished with exit code 0

Рисунок 19. Экспериментальное определение ошибок задания 3

На рисунке 20 показан результат выполнения программы с обработчиками исключений для задания 3.



```
package Lab4;  
  
import java.util.Scanner;  
  
public class Assignment3Handled {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        try {  
            System.out.print("Введите размер массива: ");  
            int n = scanner.nextInt();  
            if (n <= 0) {  
                throw new IllegalArgumentException("Размер массива должен быть положительным");  
            }  
  
            byte[] array = new byte[n];  
            for (int i = 0; i < n; i++) {  
                System.out.print("array[" + i + "] = ");  
                array[i] = scanner.nextByte();  
            }  
  
            int sum = 0;  
            for (byte value : array) {  
                sum += value;  
                if (sum < Byte.MIN_VALUE || sum > Byte.MAX_VALUE) {  
                    throw new ArithmeticException("Результат вычисления вышел за диапазон byte");  
                }  
            }  
  
            byte result = (byte) sum;  
            System.out.println("Сумма элементов byte = " + result);  
        } catch (Exception e) {  
            System.out.println("Сгенерировано исключение: " + e.getClass().getName());  
            System.out.println("Сообщение: " + e.getMessage());  
        }  
    }  
}
```

Run Assignment3Handled

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea

Введите размер массива: 3
array[0] = 300

Ошибка: введено не число или число вне диапазона byte.
Работа программы завершена.

Process finished with exit code 0

Рисунок 20. Результат выполнения программы с обработчиками задания 3

Задачи Timus

1117. Иерархия

Ограничение времени: 1.0 секунды
Ограничение памяти: 64 МБ

За долгие годы работы в СКБ Контур сложилась определённая иерархия сотрудников. Суть её заключается в том, что каждый человек, кроме рядовых работников, имеет ровно двух непосредственных подчинённых и не более одного непосредственного начальника. Рядовые же работники подчинённых не имеют (см. рисунок).



Каждому сотруднику был назначен личный номер. Естественно, никакие два сотрудника не имеют одинаковых номеров. Каждый сотрудник либо имеет максимальный номер, либо есть сотрудник с номером большим на единицу. Аналогично, либо сотрудник имеет номер 1, либо есть сотрудник с номером, меньшим на единицу. Также известно, что между любым не имеющим подчинённых (рядовым) сотрудником и не имеющим начальников сотрудником (самым главным начальником) — всегда одинаковое количество промежуточных сотрудников.

Так получилось, что у каждого сотрудника, имеющего подчинённых, номер больше номера одного из них и меньше номера другого. При этом, если у сотрудника номер больше, чем номер его непосредственного начальника, то и номера всех его подчинённых больше номера его начальника. И наоборот, если его номер меньше, то и номера подчинённых меньше.

Сложилась также и система обмена внутрикорпоративными сообщениями. Известно, что сообщение от сотрудника с номером i может быть непосредственно передано только сотруднику с номером $i - 1$ или сотруднику с номером $i + 1$. При этом передача этого сообщения происходит в тот же день (за 0 дней), если эти два сотрудника — непосредственные начальник и подчинённый. Если же между этими сотрудниками в иерархии находятся промежуточные сотрудники, то сообщение передаётся за количество дней, равное числу этих промежуточных сотрудников. Например, передача сообщения от сотрудника с номером 2 к сотруднику с номером 4 происходит следующим образом: сотрудник 2 передаёт сообщение сотруднику 3, и тот адресует его сотруднику 4. Вся эта передача сообщения занимает 1 день, так как от 2 к 3 передача происходит за 0 дней, а от 3 к 4 — за 1.

Исходные данные

В единственной строке записаны номер сотрудника, от которого передаётся сообщение, и номер сотрудника, которому передаётся сообщение — целые числа в пределах от 1 до $2^{31}-1$.

Результат

Вы должны вывести единственное число — количество дней, требующихся для передачи сообщения.

Пример

исходные данные	результат
1 5	2

Рисунок 21. Дано задачи 1000

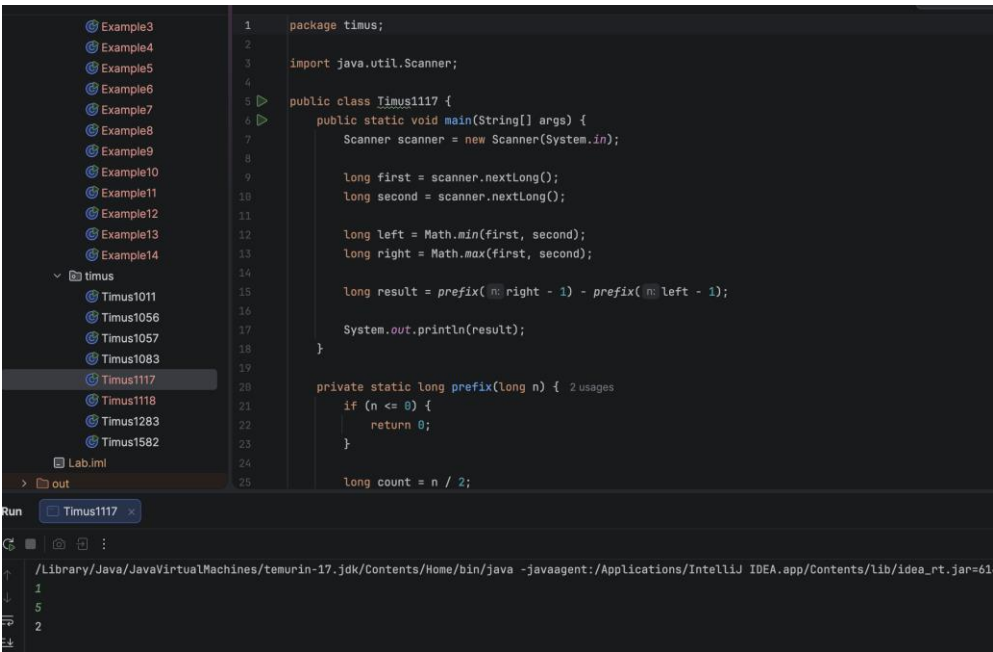


Рисунок 22. Решение задачи 1000

1118. Нетривиальные числа

Ограничение времени: 2.0 секунды

Ограничение памяти: 64 МБ

Для защиты информации в системах передачи данных через Интернет, специалистами фирмы СКБ Контур был разработан уникальный криптографический алгоритм. Главное преимущество этого алгоритма заключается в том, что в качестве ключей не требуется использовать очень большие числа; можно вполне обойтись и натуральными числами, не превосходящими миллиона. Однако, для повышения стойкости криптосистемы, рекомендуется использовать специальные числа — те самые, которые человеку психологически кажутся наименее "привычными", "правильными", "естественными". Чтобы математически определить и выделить такие числа, вводится понятие *тривиальности* натурального числа.

Тривиальностью натурального числа N будем называть отношение суммы всех его собственных делителей к самому числу. Так, например, тривиальность числа 10 равна $0.8 = (1+2+5)/10$, а тривиальность числа 20 равна $1.1 = (1+2+4+5+10)/20$. Напомним, что собственным делителем натурального числа называют любой делитель, строго меньший, чем это число.

Итак, в системе криптографической защиты информации фирмы СКБ Контур рекомендуется использовать как можно менее тривиальные числа. Вы должны составить программу, которая будет находить наименее тривиальное число в заданном диапазоне.

Исходные данные

В единственной строке записаны целые числа I и J , $1 \leq I \leq J \leq 10^6$, разделённые пробелом.

Результат

Выведите целое число N , удовлетворяющее следующим двум свойствам:

1. $I \leq N \leq J$
2. из всех целых чисел, удовлетворяющих свойству 1, N имеет наименьшую тривиальность.

Пример

исходные данные	результат
24 28	25

Автор задачи: Пётр Волков

Рисунок 23. Дано задачи 1118

```
1 package timus;
2
3 import java.util.Scanner;
4
5 public class Timus1118 {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8
9         int left = scanner.nextInt();
10        int right = scanner.nextInt();
11
12        int[] sumDivisors = new int[right + 1];
13
14        for (int divisor = 1; divisor <= right / 2; divisor++) {
15            for (int number = divisor * 2; number <= right; number += divisor) {
16                sumDivisors[number] += divisor;
17            }
18        }
19
20        int bestNumber = left;
21        int bestSum = sumDivisors[left];
22
23        for (int number = left + 1; number <= right; number++) {
24            int currentSum = sumDivisors[number];
25
26            if (currentSum < bestSum) {
27                bestSum = currentSum;
28                bestNumber = number;
29            }
30        }
31
32        System.out.println(bestNumber);
33    }
34 }
```

Run Timus1118

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents

24
28
25

Рисунок 24. Решение задачи 1118

Вывод

В ходе выполнения лабораторной работы было изучено наследование в языке Java, рассмотрена иерархия классов исключений и особенности checked и unchecked исключений. Были воспроизведены примеры 1–14 лабораторной работы, исправлены ошибки компиляции в примерах, реализованы конструкции try, catch и finally, а также использованы операторы throw и throws.

В самостоятельных заданиях были определены классы исключений, возникающие при некорректном вводе данных, отсутствии положительных элементов, обращении к несуществующему столбцу матрицы и выходе значения за границы диапазона byte. Для выявленных ошибок были созданы обработчики исключительных ситуаций с понятными сообщениями. Также были решены две задачи с сайта Timus.